

Introduzione alla libreria Numpy

Premessa: Indexing e Slicing in Python

- Indexing: Accesso ai singoli elementi di una lista (e di una sequenza in generale):
 - `s[i]` restituisce l'*i*-esimo elemento di `s` (l'indice *i* deve essere compreso tra 0 e `len(s)-1`)
- Slicing: Accesso a più elementi (contigui) di una lista (e di una sequenza in generale):
 - `s[i:j]` ritorna la sotto-lista costituita dagli elementi `s[i], s[i+1], ..., s[j-1]` di `s`, ma `s[j]` è escluso.
 - Si può usare il parametro `step`, che determina l'incremento tra ogni indice (a.k.a. `stride`). Ese.: `s[start:stop:step]`
- Sia nello Slicing che nell'Indexing un valore negativo (`-i`) usato come indice viene interpretato come `len(s)-i`. Ese.: `s[-1] == s[len(s)-1]`, `s[0:-2] == s[0:len(s)-2]`

numpy è un pacchetto per operazioni numeriche (rende Python più simile a Matlab)

```
import numpy as np
```

Gli oggetti *ndarray* di numpy rappresentano array multidimensionali e *omogenei* di oggetti di dimensione *fissa*

```
np.array([1, 2, 3, 4, 5])  
array([1, 2, 3, 4, 5])
```

Numpy Array vs List

A differenza delle liste Python, gli *ndarray* si basano su un'allocazione in memoria di un blocco contiguo di dati, quindi sono più efficienti:

```
l = list(range(10000))
a = np.array(range(10000))
def somma_numpy(a):
    return np.sum(a)

def somma_python(l):
    return sum(l)
```

```
timeit somma_python(l)
```

568 μ s $\pm$ 41.1 μ s per loop (mean $\pm$ std. dev. of 7 runs, 1000 loops each)

```
timeit somma_numpy(a)
```

9.52 μ s $\pm$ 24.8 ns per loop (mean $\pm$ std. dev. of 7 runs, 100000 loops each)

Numpy Array vs List



A differenza delle liste, gli ndarray **non** possono contenere elementi eterogenei:

```
l = [1, "ciao", False]
a = np.array([1, "ciao", False])

array(['1', 'ciao', 'False'], dtype='<U11')
```

Numpy Array vs List

Le liste di Python sono eterogenee (cioè possono contenere oggetti di qualsiasi tipo e dimensione), facilmente ridimensionabili
→ versatili

```
l = [1,2,3,4,5] # lista Python
```

Gli array sono omogenei (cioè, in un dato array sono ammessi solo oggetti tutti dello stesso tipo), invariabili nelle dimensioni, orientati alle operazioni matematiche e non inclusi in Python (è necessario importare NumPy)
→ efficienti

```
a = np.array([1,2,3,4,5]) # array Numpy
```

Numpy Array vs List



Come le liste, gli ndarray sono tipi di dato indicizzati e posso usare lo slicing:

```
l = [1,2,3,4,5] # lista Python  
a = np.array([1,2,3,4,5]) # array Numpy
```

```
l[0] == a[0]
```

```
True
```

```
l[:2]
```

```
[1, 2]
```

```
a[:2]
```

```
array([1, 2])
```

Numpy Array - Attributi



I principali attributi di un ndarray sono il tipo dei suoi elementi, la sua shape (modificabile per assegnazione o con il metodo *reshape()*), la sua size e le sue dimensioni:

```
a = np.array(range(10))  
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
a.dtype  
dtype('int64')
```

```
X = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
array([[1, 2, 3],  
       [4, 5, 6],  
       [7, 8, 9]])
```

```
X.shape  
(3, 3)  
X.size  
9  
X.ndim  
2
```

Numpy Array - Creazione

Si possono creare ndarray con caratteristiche “speciali”:

```
np.arange(10)
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
np.empty((4,2), dtype=float)
```

```
array([[ 6.92785084e-310,  4.67938787e-310],  
       [ 0.00000000e+000,  0.00000000e+000],  
       [ 0.00000000e+000,  0.00000000e+000],  
       [ 0.00000000e+000,          nan]])
```

```
np.zeros((4,2), dtype=np.int32)
```

```
array([[0, 0],  
       [0, 0],  
       [0, 0],  
       [0, 0]], dtype=int32)
```

```
np.ones((4,2), dtype=np.int32)
```

```
array([[1, 1],  
       [1, 1],  
       [1, 1],  
       [1, 1]], dtype=int32)
```

```
np.identity(3) oppure np.eye(3)
```

```
array([[ 1.,  0.,  0.],  
       [ 0.,  1.,  0.],  
       [ 0.,  0.,  1.]])
```

```
np.full((2,2), 7.0)
```

```
array([[ 7.,  7.],  
       [ 7.,  7.]])
```

```
np.random.uniform(0, 1, (2,2))
```

```
array([[0.81568445, 0.14143954],  
       [0.32903066, 0.7071234 ]])
```

Numpy Array - Operazioni

Gli ndarray possono essere moltiplicati (per degli scalari o tra loro) con:

```
X = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
array([[1, 2, 3],  
       [4, 5, 6],  
       [7, 8, 9]])
```

```
2 * X  
array([[ 2,  4,  6],  
       [ 8, 10, 12],  
       [14, 16, 18]])
```

```
np.dot(X, 2)  
array([[ 2,  4,  6],  
       [ 8, 10, 12],  
       [14, 16, 18]])
```

```
Y = np.array([[0, 0, 0], [1, 1, 1], [0, 0, 0]])  
array([[0, 0, 0],  
       [1, 1, 1],  
       [0, 0, 0]])
```

```
np.dot(X, Y)  
array([[2, 2, 2],  
       [5, 5, 5],  
       [8, 8, 8]])
```

```
X * Y (element-wise)  
array([[0, 0, 0],  
       [4, 5, 6],  
       [0, 0, 0]])
```

Numpy Array - Operazioni

Altri esempi di operazioni "element-wise":

- $x + y$
- $x - y$
- x / y
- `np.sqrt(x)`

Sommatoria:

```
x = np.array([[1, 2], [3, 4]])  
print(np.sum(x))    # Compute sum of all elements; prints "10"  
print(np.sum(x, axis=0)) # Compute sum of each column; prints "[4 6]"  
print(np.sum(x, axis=1)) # Compute sum of each row; prints "[3 7]"
```

Numpy Array - Operazioni

Gli ndarray possono essere trasposti con:

```
X = np.array([[1, 2, 3], [4, 5, 6],  
              [7, 8, 9]])  
array([[1, 2, 3],  
       [4, 5, 6],  
       [7, 8, 9]])
```

```
X.T  
array([[1, 4, 7],  
       [2, 5, 8],  
       [3, 6, 9]])
```

```
np.transpose(X)  
array([[1, 4, 7],  
       [2, 5, 8],  
       [3, 6, 9]])
```

Numpy Array - Indicizzazione



Si può accedere agli elementi di un ndarray con:

```
a = np.array([1, 2, 3, 4, 5])
```

```
a[0]
```

```
1
```

```
a[-1]
```

```
5
```

```
a = np.array([[1, 2, 3], [3, 4, 5], [4, 5, 6]])
```

```
array([[1, 2, 3],  
       [3, 4, 5],  
       [4, 5, 6]])
```

```
a[..., 1] oppure a[:, 1]
```

```
array([2, 4, 5])
```

```
a[2, ...] oppure a[2, :]
```

```
array([4, 5, 6])
```

```
a[0:2, 0:2]
```

```
array([[1, 2],  
       [3, 4]])
```

Quando si opera con gli array è essenziale sapere se i dati vengono copiati in un nuovo array o no:

- **Nessuna copia:** Le istruzioni di assegnazione come `arr2 = arr1` non copiano i dati, creano solo un alias
- **Copia profonda:** Il metodo `copy()` crea una copia completa dell'array e dei dati.
Ese.:
 - `a = np.array([1,2,3,4,5])`
 - `c = np.copy(a)`
- **Vista o copia superficiale:** lo slicing produce una "vista", cioè un nuovo array *con dati condivisi*

Esempio di "Vista"



```
a = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])  
array([[ 1,  2,  3,  4],  
       [ 5,  6,  7,  8],  
       [ 9, 10, 11, 12]])
```

```
b = a[:2, 1:3]  
array([[2, 3],  
       [6, 7]])
```

```
b[0,0] = 99
```

```
a  
array([[ 1, 99,  3,  4],  
       [ 5,  6,  7,  8],  
       [ 9, 10, 11, 12]])
```